Descripción Técnica del Algoritmo Sargon en Challenger

Proyecto Challenger

28 de septiembre de 2026

Resumen

Este documento describe formalmente el algoritmo de ajedrez denominado Sargon implementado en el proyecto `challenger`. El motor sigue un enfoque determinista clásico basado en evaluación heurística de posiciones, búsqueda adversarial minimax, poda alpha–beta, búsqueda de quiescence, ordenamiento táctico de jugadas y tabla de transposición. No utiliza aprendizaje por refuerzo, redes neuronales ni actualización estadística derivada de partidas previas.

Índice

1. Planteo general

Dada una posición de ajedrez s, Challenger debe seleccionar una jugada

$$m^* \in \mathcal{M}(s),$$

donde $\mathcal{M}(s)$ es el conjunto de movimientos legales en s.

Cada movimiento m induce una nueva posición

$$s' = T(s, m),$$

donde T es la función de transición del juego.

El movimiento seleccionado es aquel que maximiza el valor minimax desde la perspectiva del jugador raíz:

$$m^* = \arg \max_{m \in \mathcal{M}(s)} V(T(s, m), d - 1),$$

donde d es el número de plies configurado mediante el argumento `-plies`. El valor por defecto es

$$d = 2.$$

Con dos plies, el motor analiza conceptualmente:

$$\text{jugada propia} \rightarrow \text{respuesta rival},$$

y posteriormente puede extender selectivamente líneas tácticas mediante búsqueda de quiescence.

2. Perspectiva de evaluación

Sea

$$r \in \{\text{White}, \text{Black}\}$$

el color del jugador raíz.

Toda evaluación se expresa desde su perspectiva:

$$E(s, r) > 0$$

indica ventaja para Challenger, mientras que

$$E(s, r) < 0$$

indica ventaja para el adversario.

Esta convención permite formular minimax directamente, sin alternar signos en cada nivel como sería habitual en una implementación Negamax.

3. Función de evaluación estática

La evaluación bruta de una posición se define como

$$E_{\text{raw}}(s, r) = M(s, r) + A(s, r) + P(s, r) + K(s, r),$$

donde:

- M: material;
- A: actividad, desarrollo y control del centro;

- P: estructura de peones;
- K: seguridad del rey.

El valor bruto se normaliza mediante:

$$\boxed{E(s,r) = \tanh\left(\frac{E_{\text{raw}}(s,r)}{8}\right)}$$

de modo que, para posiciones no terminales,

$$-1 < E(s,r) < 1.$$

La función hiperbólica está dada por

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}.$$

Algunos valores ilustrativos son:

$$E_{\text{raw}} = 0 \Rightarrow E = 0,$$
$$E_{\text{raw}} = 4 \Rightarrow \tanh(0{,}5) \approx 0{,}462,$$
$$E_{\text{raw}} = 8 \Rightarrow \tanh(1) \approx 0{,}762,$$
$$E_{\text{raw}} = 16 \Rightarrow \tanh(2) \approx 0{,}964.$$

4. Evaluación material

Los valores de las piezas son:

Pieza	Valor
Peón	1,0
Caballo	3,2
Alfil	3,3
Torre	5,0
Dama	9,0
Rey	0

Por tanto,

$$M(s,r) = \sum_{p \in r} v(p) - \sum_{p \in \bar{r}} v(p).$$

El rey no recibe valor material porque su pérdida no se modela como captura, sino como jaque mate, que se trata separadamente como estado terminal.

5. Actividad de piezas

Para cada pieza p, situada en una casilla determinada, se calcula la cantidad de casillas atacadas:

$$a(p) = |\,\text{Attacks}(p)|.$$

La contribución de actividad es

$$A_{\text{mobility}}(p) = \sigma(p)\,0{,}015\,a(p),$$

donde

$$\sigma(p) = \begin{cases} +1, & p \text{ pertenece a Challenger,} \\ -1, & p \text{ pertenece al rival.} \end{cases}$$

Esta métrica representa actividad geométrica más que movilidad legal estricta.

6. Control del centro

Se considera el centro principal

$$C = \{d4, e4, d5, e5\}.$$

Si una pieza ocupa una casilla central:

$$A_C(p) = \sigma(p) \begin{cases} 0{,}10, & p \in \{P, N, B\}, \\ 0{,}04, & \text{otra pieza.} \end{cases}$$

También se considera un centro extendido:

$$C_e = \{c3, d3, e3, f3, c4, f4, c5, f5, c6, d6, e6, f6\},$$

con contribución

$$A_{C_e}(p) = 0{,}025\,\sigma(p).$$

7. Desarrollo de piezas menores

Los caballos y alfiles que permanecen en sus casillas iniciales son penalizados.
Para los caballos en

$$b1,\ g1,\ b8,\ g8$$

se aplica

$$0{,}08.$$

Para los alfiles en

$$c1,\ f1,\ c8,\ f8$$

se aplica

$$0{,}06.$$

Por tanto:

$$A_{\text{dev}}(p) = \begin{cases} -0{,}08\sigma(p), & \text{caballo no desarrollado,} \\ -0{,}06\sigma(p), & \text{alfil no desarrollado,} \\ 0, & \text{otro caso.} \end{cases}$$

8. Pareja de alfiles

Si un color conserva al menos dos alfiles:

$$n_B(c) \geq 2,$$

se asigna una bonificación de

$$0{,}12.$$

Así:

$$A_{\text{bishop}} = 0{,}12\,[I(n_B(r) \geq 2) - I(n_B(\bar{r}) \geq 2)]\,,$$

donde I es la función indicadora.

9. Estructura de peones

9.1. Peones doblados

Sea $n_f(c)$ el número de peones del color c en la columna f.
La cantidad de peones excedentes en columnas con duplicación es

$$D_c = \sum_f \text{máx}(0, n_f(c) - 1).$$

La penalización es

$$0{,}08 D_c.$$

Desde la perspectiva de Challenger:

$$P_{\text{double}} = -0{,}08 D_r + 0{,}08 D_{\bar{r}}.$$

9.2. Peones aislados

Sea F_c el conjunto de columnas ocupadas por al menos un peón del color c.
Una columna f se considera aislada si

$$f - 1 \notin F_c \quad \wedge \quad f + 1 \notin F_c.$$

Sea I_c la cantidad de columnas con peones aislados. La penalización es

$$0{,}04 I_c.$$

Por tanto:

$$P_{\text{isolated}} = -0{,}04 I_r + 0{,}04 I_{\bar{r}}.$$

La evaluación total de la estructura de peones es:

$$\boxed{P = -0{,}08 D_r - 0{,}04 I_r + 0{,}08 D_{\bar{r}} + 0{,}04 I_{\bar{r}}}$$

10. Seguridad del rey

10.1. Jaque

Si el rey del jugador raíz está en jaque, la contribución es

$$-0{,}35.$$

Si quien está en jaque es el adversario, la contribución es

$$+0{,}35.$$

10.2. Escudo de peones

Se inspeccionan las tres casillas inmediatamente delante del rey:

$$(k_f - 1, k_r + d), \quad (k_f, k_r + d), \quad (k_f + 1, k_r + d),$$

con

$$d = \begin{cases} +1, & \text{blancas}, \\ -1, & \text{negras}. \end{cases}$$

Cada peón propio presente en esas casillas suma

$$0{,}05.$$

Así:

$$K_{\text{shield}} = 0{,}05S,$$

donde

$$0 \leq S \leq 3.$$

La bonificación máxima es entonces

$$0{,}15.$$

10.3. Casillas peligrosas alrededor del rey

Sea $R(k)$ el conjunto de casillas adyacentes al rey.
Se define

$$D_c = |\{q \in R(k_c) : q \text{ está atacada por el enemigo}\}|.$$

Cada casilla peligrosa produce una penalización de

$$0{,}035.$$

Por tanto:

$$K_{\text{danger}} = -0{,}035D_r + 0{,}035D_{\bar{r}}.$$

La seguridad del rey queda:

$$K = K_{\text{check}} + K_{\text{shield}} + K_{\text{danger}}.$$

11. Evaluación total

La función de evaluación puede resumirse como

$$\boxed{E_{\text{raw}} = M + A_{\text{mobility}} + A_{\text{center}} + A_{\text{development}} + A_{\text{bishop}} + P_{\text{double}} + P_{\text{isolated}} + K_{\text{check}} + K_{\text{shield}} + K_{\text{danger}}}$$

y:

$$\boxed{E = \tanh\left(\frac{E_{\text{raw}}}{8}\right)}$$

12. Estados terminales

La evaluación heurística no se utiliza si la posición ya terminó.

12.1. Jaque mate

Si el jugador al turno está en mate:

$$E(s, r) = \begin{cases} -1, & \text{turn}(s) = r, \\ +1, & \text{turn}(s) \neq r. \end{cases}$$

Por tanto:

$$\boxed{\text{victoria} = +1}$$

y

$$\boxed{\text{derrota} = -1}.$$

12.2. Tablas

Para cualquier estado reconocido como tablas:

$$\boxed{E(s, r) = 0}.$$

La lógica de Challenger contempla, mediante `python-chess`, los casos de:

- rey ahogado;
- material insuficiente;
- regla automática de 75 movimientos;
- repetición quíntuple automática;
- triple repetición reclamable;
- regla de 50 movimientos reclamable.

En Challenger las tablas reclamables se reclaman automáticamente.

13. Minimax

Sea

$$V(s, d, \alpha, \beta)$$

el valor de una posición a profundidad d.

Si la partida terminó:

$$V(s, d) = E(s).$$

Si

$$d = 0,$$

el algoritmo entra en búsqueda de quiescence.

Para $d > 0$, si juega Challenger:

$$\boxed{V(s, d) = \max_{m \in \mathcal{M}(s)} V(T(s, m), d - 1)}$$

Si juega el adversario:

$$\boxed{V(s, d) = \min_{m \in \mathcal{M}(s)} V(T(s, m), d - 1)}$$

14. Ejemplo de minimax a dos plies

Supóngase que Challenger dispone de tres jugadas A, B y C con respuestas rivales evaluadas de la siguiente forma:

$$A \to \{0{,}4, 0{,}1, 0{,}5\},$$

$$B \to \{0{,}3, 0{,}25, 0{,}2\},$$

$$C \to \{0{,}8, -0{,}4, 0{,}7\}.$$

El adversario elige la peor continuación para Challenger:

$$V(A) = \min(0{,}4, 0{,}1, 0{,}5) = 0{,}1,$$

$$V(B) = \text{mín}(0{,}3, 0{,}25, 0{,}2) = 0{,}2,$$
$$V(C) = \text{mín}(0{,}8, -0{,}4, 0{,}7) = -0{,}4.$$

Challenger elige:

$$\text{máx}(0{,}1, 0{,}2, -0{,}4) = 0{,}2,$$

y por tanto:

$$\boxed{m^* = B}.$$

15. Poda alpha–beta

Minimax puro requiere aproximadamente:

$$O(b^d),$$

donde b es el factor de ramificación medio y d la profundidad.

Con

$$b \simeq 35$$

y

$$d = 4,$$

el número teórico de nodos sería:

$$35^4 = 1\,500\,625.$$

Alpha–beta mantiene dos cotas:

$$\alpha = \text{mejor valor garantizado por MAX},$$

$$\beta = \text{mejor valor garantizado por MIN}.$$

Si:

$$\boxed{\alpha \geq \beta},$$

la rama restante se poda.

Para MAX:

$$\alpha \leftarrow \text{máx}(\alpha, V).$$

Para MIN:

$$\beta \leftarrow \text{mín}(\beta, V).$$

La búsqueda se inicia con valores amplios:

$$\alpha = -2, \qquad \beta = +2.$$

Con buen ordenamiento de movimientos, la complejidad práctica puede acercarse a

$$O(b^{d/2}),$$

aunque este resultado depende fuertemente del orden de exploración.

16. Ordenamiento de jugadas

La eficiencia de alpha–beta depende del orden en que se examinan las jugadas.
Se asigna a cada movimiento una prioridad heurística $O(m)$.

16.1. Promoción

Si la jugada promociona:

$$O(m) \mathrel{+}= 20 + v(\text{pieza promovida}).$$

Por ejemplo, promoción a dama:

$$20 + 9 = 29.$$

16.2. Captura

Para una captura:

$$O(m) \mathrel{+}= 10 + v(\text{víctima}) - 0{,}1v(\text{atacante}).$$

Esta fórmula es una aproximación al criterio MVV–LVA (*Most Valuable Victim / Least Valuable Attacker*).

Ejemplo: peón captura dama:

$$10 + 9 - 0{,}1(1) = 18{,}9.$$

Dama captura peón:

$$10 + 1 - 0{,}1(9) = 10{,}1.$$

16.3. Jaque

Una jugada que da jaque suma:

$$6.$$

Entonces:

$$\boxed{O(m) = O_{\text{promotion}} + O_{\text{capture}} + O_{\text{check}}}$$

Las jugadas se ordenan de mayor a menor prioridad.

17. Efecto horizonte

Si la búsqueda se corta exactamente después de una captura favorable, puede sobrevalorar una secuencia táctica cuyo desenlace real ocurre inmediatamente después del horizonte.

Este fenómeno se conoce como:

$$\boxed{\text{horizon effect}}.$$

Por ello, cuando la profundidad principal llega a cero, Challenger no evalúa inmediatamente la posición sino que entra en búsqueda de quiescence.

18. Búsqueda de quiescence

La profundidad de quiescence por defecto es:

$$q = 2.$$

En posiciones normales se continúan solamente movimientos forzantes:

$$\mathcal{F}(s) = \{\text{capturas}, \text{promociones}, \text{jaques}\}.$$

Si el rey está en jaque:

$$\mathcal{F}(s) = \mathcal{M}(s),$$

porque cualquier respuesta legal puede ser necesaria.

18.1. Stand-pat

Se calcula primero:

$$S = E(s),$$

denominado *stand-pat*.

Si juega Challenger:

$$Q(s,q) = \max\left(S, \max_{m \in \mathcal{F}(s)} Q(T(s,m), q-1)\right).$$

Si juega el rival:

$$Q(s,q) = \min\left(S, \min_{m \in \mathcal{F}(s)} Q(T(s,m), q-1)\right).$$

También se utiliza poda alpha–beta dentro de la búsqueda de quiescence.

19. Profundidad efectiva

Por esta razón:

```
-plies 2
```

no implica que solo se inspeccionen dos medios movimientos.

La búsqueda principal tiene:

$$2 \text{ plies},$$

pero una línea táctica puede extenderse aproximadamente hasta:

$$2 + q.$$

Con

$$q = 2,$$

la profundidad táctica efectiva puede alcanzar aproximadamente:

$$4 \text{ plies},$$

aunque solo en ramas tácticamente forzadas.

20. Selección final

Para cada movimiento raíz m_i se calcula:

$$v_i = V(T(s, m_i), d-1).$$

Se define:

$$v^* = \max_i v_i.$$

Luego se forma:

$$B = \{m_i : |v_i - v^*| \leq 10^{-12}\}.$$

Si:

$$|B| = 1,$$

se elige el único mejor movimiento.

Si:

$$|B| > 1,$$

Challenger selecciona aleatoriamente:

$$m^* \sim U(B).$$

La aleatoriedad solo se aplica entre jugadas evaluadas como equivalentes.

21. Tabla de transposición

El motor mantiene una tabla:

$$TT[k] = V.$$

La clave utilizada puede representarse como:

$$k = (FEN(s), H(s), d, r),$$

donde:

- $FEN(s)$ es la representación FEN;
- $H(s)$ es la historia de movimientos;
- d es la profundidad restante;
- r es el color raíz.

La historia se representa como:

$$H(s) = (m_1, m_2, \ldots, m_n).$$

La incorporación de historia es necesaria porque una posición con el mismo FEN puede tener diferente situación respecto de las reglas de repetición.

Esta estrategia es conservadora: evita reutilizaciones incorrectas, pero reduce la probabilidad de colisiones útiles entre posiciones equivalentes alcanzadas por órdenes distintos de jugadas.

22. Conteo de nodos

Cada llamada de minimax o quiescence incrementa un contador:

$$N \leftarrow N + 1.$$

El resultado global de una búsqueda puede representarse como:

$$R = (m^*, v^*, N, t),$$

donde:

- m^* es el movimiento elegido;
- v^* es su evaluación;
- N es la cantidad de nodos examinados;
- t es el tiempo consumido.

23. Resumen matemático del algoritmo

El proceso completo puede expresarse como:

$$s_0 \xrightarrow{\mathcal{M}} m_1, \ldots, m_n.$$

Para cada movimiento:

$$s_i = T(s_0, m_i),$$

y:

$$v_i = \text{Minimax}(s_i, d-1, -2, +2).$$

Cuando la profundidad llega a cero:

$$d = 0 \Rightarrow \text{Quiescence}(s, q).$$

La búsqueda de quiescence termina finalmente en:

$$E(s) = \tanh\left(\frac{M + A + P + K}{8}\right),$$

salvo posiciones terminales:

$$E = \begin{cases} +1, & \text{victoria}, \\ 0, & \text{tablas}, \\ -1, & \text{derrota}. \end{cases}$$

Finalmente:

$$\boxed{m^* = \arg\max_{m_i} v_i}.$$

24. Ausencia de aprendizaje

El algoritmo Sargon de Challenger no utiliza aprendizaje.
No existen:

- ecuación de Bellman;
- Q-learning;
- redes neuronales;
- retropropagación;
- actualización de pesos después de una partida;
- memoria histórica de valor posicional aprendida.

Cada posición se resuelve de nuevo mediante:

$$\boxed{\text{heurística} + \text{búsqueda}}$$

Por tanto, jugar un gran número de partidas no hace al motor automáticamente más fuerte.
Su rendimiento mejora fundamentalmente mediante:

$$\text{mayor profundidad},$$

$$\text{mejor función de evaluación},$$

o

$$\text{búsqueda más eficiente}.$$

25. Diferencia respecto de un enfoque Bellman

Sargon aproxima:

$$V(s) \approx \text{MinimaxDepthD}(s)$$

a partir del análisis directo de la posición actual.

En cambio, un esquema típico de Q-learning sería:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right].$$

Sargon intenta responder:

> £Qué ocurre si analizo esta posición?

Un sistema Bellman intenta responder:

> £Qué valor aprendí históricamente para situaciones y acciones semejantes?

Ambos enfoques pertenecen a paradigmas conceptualmente diferentes.

26. Limitación principal

La principal limitación del Sargon actual es la profundidad de búsqueda.

Con:

$$d = 2,$$

el motor suele manejar razonablemente:

- tácticas inmediatas;
- capturas;
- jaques;
- promociones;
- respuestas inmediatas del rival;
- ciertos intercambios gracias a quiescence.

Sin embargo, no puede descubrir de forma sistemática combinaciones cuya justificación aparezca varios plies más adelante.

Puede existir una secuencia:

$$m_1 \to m_2 \to m_3 \to m_4 \to m_5 \to m_6,$$

en la cual la ventaja solo se manifiesta en m_6.

Si la posición después de m_2 no contiene movimientos forzantes que activen quiescence, una búsqueda con dos plies no alcanza esa consecuencia.

27. Relación con el Sargon histórico

Conviene distinguir:

$$\text{Sargon histórico} \neq \text{Sargon de Nyx} \neq \text{Sargon de Challenger}.$$

El Sargon histórico de Dan y Kathe Spracklen fue diseñado para máquinas con recursos extremadamente limitados.

El Sargon de Challenger conserva la filosofía central:

$$\boxed{\text{generar} \rightarrow \text{evaluar} \rightarrow \text{buscar adversarialmente}}$$

pero utiliza herramientas modernas como:

- validación legal mediante `python-chess`;
- minimax con poda alpha–beta;
- búsqueda de quiescence;
- ordenamiento táctico de jugadas;
- tabla de transposición;
- evaluación normalizada;
- detección completa de condiciones reglamentarias de finalización.

Por tanto, una descripción técnicamente precisa es:

> Motor de ajedrez Sargon-style basado en evaluación heurística y búsqueda minimax alpha–beta, sin aprendizaje, enriquecido con quiescence y ordenamiento táctico.

28. Extensiones naturales

Manteniendo completamente el paradigma Sargon y sin introducir aprendizaje neuronal ni Bellman, las extensiones de mayor impacto probable serían:

- iterative deepening;
- aspiration windows;
- killer moves;
- history heuristic;
- tabla de transposición con cotas EXACT, LOWER y UPPER;
- principal variation search;
- mejor ordenamiento de jugadas;
- extensiones selectivas de jaques o recapturas;
- reducción de movimientos tardíos, con especial cuidado para preservar la estabilidad táctica.

Estas técnicas permitirían aumentar la profundidad efectiva de análisis sin modificar la naturaleza fundamental del motor.

29. Conclusión

El Sargon de Challenger es un motor clásico de búsqueda adversarial. Su decisión surge de la combinación de una evaluación heurística explícita con una búsqueda minimax optimizada.

En forma compacta:

$$\boxed{\text{Sargon} = \text{Evaluación heurística} + \text{Minimax} + \alpha\beta + \text{Quiescence} + \text{Ordenamiento} + \text{Transposiciones}}$$

No aprende de partidas anteriores. Cada jugada se decide mediante análisis directo del árbol de juego disponible a la profundidad configurada.